TD : ressources locales et préférence d'affichage

Objectif

À partir de l'interface de la branche `jour1-interface`, vous allez ajouter le contenu de la branche `jour1-ressources-stockage` : un bouton d'affichage compact, une section pédagogique et une préférence conservée par le navigateur.

À la fin du TD, l'interface utilise toujours les fichiers Bootstrap présents dans le projet. Les documents ajoutés restent temporaires. **Seul le choix d'affichage est mémorisé** ; aucun fichier ni document n'est envoyé au serveur.

Avant de commencer

Suivez le guide de démarrage pour cloner le projet, puis ouvrez le dossier `app-frontend-documents` dans VS Code. Le dossier contient `index.html`, `assets/` et `README.md`.

Dans **Terminal > Nouveau terminal**, vérifiez la branche :

```
git branch --show-current
```

La réponse doit être `jour1-interface`. Si vous devez y revenir, sauvegardez d'abord votre travail puis lancez `git switch jour1-interface`. Si Git refuse le changement à cause de fichiers modifiés, demandez au formateur comment les conserver ; ne les supprimez pas.

Ouvrez `index.html` dans un navigateur. La page affiche trois documents fictifs et ne possède pas encore le bouton **Affichage compact**. Dans les étapes suivantes, enregistrez chaque fichier après modification avec **Ctrl+S**.

1. Comprendre les ressources locales

Dans VS Code, ouvrez `index.html`. Dans la section `<head>`, recherchez la ligne :

```
  <link rel="stylesheet" href="assets/vendor/bootstrap/bootstrap.min.css">
```

Ajoutez **juste avant** cette ligne le commentaire suivant :

```
  <!-- Bootstrap est fourni dans le projet : aucun CDN n'est nécessaire. -->
```

Le chemin `assets/vendor/bootstrap/` désigne les fichiers Bootstrap livrés avec le projet. Un **CDN** est un service qui pourrait fournir ces fichiers sur Internet ; ici, le navigateur charge les copies locales. Il n'est pas nécessaire de télécharger Bootstrap pendant cet exercice.

2. Prévoir le script de la préférence

Toujours dans `<head>` de `index.html`, repérez la ligne qui charge `assets/js/app.js`. Ajoutez la ligne suivante **juste après** :

```
  <script src="assets/js/securite.js" defer></script>
```

Vous créerez ce fichier à l'étape 6. Les scripts sont chargés avec `defer` et exécutés dans l'ordre où ils apparaissent : `app.js` prépare notamment `documentList`, que `securite.js` utilisera ensuite. Conservez cet ordre.

3. Ajouter un lien dans la navigation

Dans `index.html`, repérez le lien **Ajouter un document** dans la liste `<ul class="navbar-nav ...">`. Ajoutez la ligne suivante juste après :

```
          <li class="nav-item"><a class="nav-link" href="#securite">Sécurité</a></li>
```

`href="#securite"` pointe vers une section de la même page. Vous ajouterez son identifiant à l'étape 5.

4. Ajouter le bouton d'affichage compact

Dans `index.html`, recherchez cette ligne dans l'en-tête de la liste :

```
            <span id="document-count" class="count-label">3 documents</span>
```

Remplacez **cette ligne uniquement** par le bloc suivant :

```
            <div class="d-flex flex-wrap align-items-center gap-3">
              <span id="document-count" class="count-label">3 documents</span>
              <div class="form-check form-switch mb-0">
                <input class="form-check-input" type="checkbox" role="switch" id="compact-mode">
                <label class="form-check-label small" for="compact-mode">Affichage compact</label>
              </div>
            </div>
```

Le compteur garde son identifiant `document-count`, utilisé par `app.js`. La case `compact-mode` représente le choix de l'utilisateur ; son étiquette permet aussi de l'activer en cliquant sur le texte. Les classes Bootstrap organisent les éléments sur la même ligne lorsque la place le permet.

5. Ajouter la section de démonstration

Dans `index.html`, repérez la balise `</main>` à la fin du contenu principal. Collez le bloc suivant **juste avant `</main>`**, après le `</div>` qui termine la rangée contenant la liste et le formulaire :

```
  <!-- Cette zone sert uniquement aux démonstrations de la formation. -->
  <section id="securite" class="card app-card mt-4" aria-labelledby="securite-title">
    <div class="card-body p-4">
      <h2 id="securite-title" class="section-title mb-2">Démonstrations de sécurité</h2>
      <p class="section-subtitle">Ces exemples sont visibles dans les outils de développement du navigateur.</p>
      <div class="row g-4 mt-1">
```

```
          <div class="col-12 col-md-6">
            <h3 class="demo-title">Préférence d’affichage</h3>
            <p class="demo-text">Le bouton « Affichage compact » mémorise votre choix dans le navigateur.</p>
            <p id="storage-message" class="demo-status mb-0" role="status" aria-live="polite"></p>
          </div>
          <div class="col-12 col-md-6">
            <h3 class="demo-title">Ressources locales</h3>
            <p class="demo-text">Bootstrap est fourni dans le projet. L’interface fonctionne sans connexion à un CDN.</p>
          </div>
        </div>
      </div>
    </section>
```

L'identifiant `securite` correspond au lien de navigation ajouté précédemment. Le paragraphe `storage-message` permettra au JavaScript d'indiquer si la préférence a pu être mémorisée. `role="status"` et `aria-live="polite"` aident à annoncer ses changements aux technologies d'assistance.

6. Définir le style

Ouvrez `assets/css/style.css`. Recherchez le sélecteur `.document-icon {` et collez le code suivant **juste avant** ce sélecteur :

```
/* Une préférence simple : réduire l'espace autour des documents. */
.document-list.compact .document-item {
  padding-top: .55rem;
  padding-bottom: .55rem;
}

.demo-title {
  font-size: .95rem;
  font-weight: 700;
}

.demo-text, .demo-status {
  color: var(--muted);
  font-size: .85rem;
}
```

Quand la classe `compact` est présente sur la liste, le premier sélecteur réduit l'espace vertical des documents. Les autres règles donnent leur apparence aux textes de la nouvelle section. `--muted` est une couleur déjà définie dans le fichier.

7. Mémoriser et appliquer le choix

Dans `assets/js/`, créez le fichier **`securite.js`**. Copiez-y tout le code suivant et enregistrez :

```
// Démonstration localStorage : mémoriser une préférence non sensible.
const compactButton = document.querySelector('#compact-mode');
const storageMessage = document.querySelector('#storage-message');
const storageKey = 'formation_affichage_compact';

function appliquerAffichage(compact) {
  documentList.classList.toggle('compact', compact);
  compactButton.checked = compact;
}

try {
  // La valeur reste dans ce navigateur après un rechargement.
  appliquerAffichage(localStorage.getItem(storageKey) === 'oui');
```

```
  storageMessage.textContent = 'Choix conservé dans ce navigateur.';
} catch (error) {
  // Certains navigateurs ou paramètres peuvent bloquer le stockage.
  storageMessage.textContent = 'Le stockage du navigateur est indisponible.';
}

compactButton.addEventListener('change', () => {
  appliquerAffichage(compactButton.checked);
  try {
    localStorage.setItem(storageKey, compactButton.checked ? 'oui' : 'non');
    storageMessage.textContent = 'Choix conservé dans ce navigateur.';
  } catch (error) {
    storageMessage.textContent = 'Choix appliqué, mais non mémorisé par le navigateur.';
  }
});
```

`querySelector` retrouve les éléments créés dans `index.html`. La fonction `appliquerAffichage` ajoute ou retire la classe `compact` et met à jour la case. Au chargement, `getItem` lit le choix précédent ; lors d'un changement, `setItem` enregistre la valeur `oui` ou `non` sous la clé `formation_affichage_compact`.

Les blocs `try...catch` permettent d'afficher un message si le navigateur refuse l'accès au stockage. `localStorage` conserve une petite valeur dans le navigateur pour le même site ; il ne remplace pas une base de données. N'y stockez pas de mot de passe ou de jeton de connexion : les scripts exécutés sur la page peuvent y accéder.

8. Vérifier le résultat

1. Rechargez `index.html`. Le lien **Sécurité**, le bouton **Affichage compact** et la section de démonstration doivent apparaître.
2. Activez le bouton : les lignes de documents deviennent plus compactes.
3. Rechargez la page : le bouton reste actif et les lignes restent compactes si le navigateur autorise le stockage.
4. Désactivez le bouton puis rechargez : l'affichage normal doit revenir.
5. Ajoutez un document avec le formulaire, puis rechargez : la ligne ajoutée disparaît. Les trois documents fictifs sont à nouveau affichés.

Pour voir la valeur stockée, ouvrez les outils de développement du navigateur, puis **Application > Stockage local** et recherchez la clé `formation_affichage_compact`. L'accès à `localStorage` peut varier lorsque la page est ouverte directement en `file://` ; si le navigateur le bloque, demandez au formateur d'utiliser un serveur web local pour la démonstration.

9. Comparer avec la correction

La branche `jour1-ressources-stockage` contient le résultat attendu. Pour comparer sans changer de branche et sans écraser votre travail, exécutez dans le terminal du projet :

```
git fetch origin
git diff origin/jour1-ressources-stockage -- index.html assets/css/style.css assets/js/app.js assets/js/securite.js
```

Une fois les fichiers enregistrés, aucune différence ne doit apparaître pour `index.html`, `style.css` et `app.js`. Comme le nouveau fichier `securite.js` n'est pas encore suivi par Git, la comparaison peut le signaler comme absent même s'il existe sur votre disque. Pour le vérifier sans l'ajouter à Git, affichez la correction avec :

```
git show origin/jour1-ressources-stockage:assets/js/securite.js
```

Le README de la branche de correction présente également cette étape. Votre guide de démarrage et ce TD sont des documents d'accompagnement.

En cas de difficulté

Situation	Vérification
Le lien Sécurité ne descend pas à la section	Vérifiez `href="#securite"` et `id="securite"`.
Le bouton apparaît mais ne change rien	Vérifiez que `securite.js` existe, que son chemin est correct et qu'il est chargé après `app.js`.
Le message de stockage reste vide	Vérifiez `id="storage-message"`, enregistrez les fichiers et consultez la console du navigateur.
Le choix ne survit pas au rechargement	Vérifiez les paramètres du navigateur et le message de stockage. En `file://`, essayez un serveur local avec le formateur.
Un document ajouté disparaît au rechargement	C'est le comportement attendu : cet exercice conserve seulement la préférence d'affichage.